

multicollinearity__toy__ex__tk

September 26, 2025

```
[1]: import numpy as np
import pandas as pd
from statsmodels.formula.api import ols
from plotnine import ggplot, aes, geom_point, labs
from scipy.stats import linregress
```

1 Multicollinearity Demo

```
[2]: ### multicollinearity

# Uncorrelated predictor variables
X1 = [4, 4, 4, 4, 6, 6, 6, 6]
X2 = [2, 2, 3, 3, 2, 2, 3, 3]
Y = [42, 39, 48, 51, 49, 53, 61, 60]
```

```
[3]: # Creating DataFrame
df = pd.DataFrame({
    'X1': X1,
    'X2': X2,
    'Y': Y
})

# Plot X1 vs X2
plot = (ggplot(df, aes(x='X1', y='X2')) +
        geom_point() +
        labs(x='X1', y='X2', title='Scatter Plot of X1 vs X2'))
plot
```



```
[4]: # Correlation between X1 and X2
correlation = np.corrcoef(X1, X2)[0, 1]
print(f'Correlation between X1 and X2: {correlation}')
```

Correlation between X1 and X2: 0.0

```
[5]: # Linear model Y ~ X1 + X2
model1 = ols('Y ~ X1 + X2', data=df).fit()
model1.summary()
```

/opt/anaconda3/lib/python3.13/site-packages/scipy/stats/_axis_nan_policy.py:430:
UserWarning: `kurtosistest` p-value may be inaccurate with fewer than 20
observations; only n=8 observations were given.

[5]:

Dep. Variable:	Y	R-squared:	0.958
Model:	OLS	Adj. R-squared:	0.941
Method:	Least Squares	F-statistic:	57.06
Date:	Sun, 21 Sep 2025	Prob (F-statistic):	0.000361
Time:	23:25:58	Log-Likelihood:	-14.511
No. Observations:	8	AIC:	35.02
Df Residuals:	5	BIC:	35.26
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.3750	4.740	0.079	0.940	-11.811	12.561
X1	5.3750	0.664	8.097	0.000	3.669	7.081
X2	9.2500	1.328	6.968	0.001	5.837	12.663
Omnibus:	2.902		Durbin-Watson:	2.773		
Prob(Omnibus):	0.234		Jarque-Bera (JB):	0.878		
Skew:	-0.108		Prob(JB):	0.645		
Kurtosis:	1.391		Cond. No.	42.1		

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[6]: # Linear model Y ~ X1
model_X1 = ols('Y ~ X1', data=df).fit()
model_X1.summary()
```

/opt/anaconda3/lib/python3.13/site-packages/scipy/stats/_axis_nan_policy.py:430:
UserWarning: `kurtosistest` p-value may be inaccurate with fewer than 20
observations; only n=8 observations were given.

[6]:

Dep. Variable:	Y	R-squared:	0.550
Model:	OLS	Adj. R-squared:	0.476
Method:	Least Squares	F-statistic:	7.347
Date:	Sun, 21 Sep 2025	Prob (F-statistic):	0.0351
Time:	23:25:58	Log-Likelihood:	-23.995
No. Observations:	8	AIC:	51.99
Df Residuals:	6	BIC:	52.15
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	23.5000	10.111	2.324	0.059	-1.242	48.242
X1	5.3750	1.983	2.711	0.035	0.523	10.227
Omnibus:	3.389		Durbin-Watson:	1.815		
Prob(Omnibus):	0.184		Jarque-Bera (JB):	0.935		
Skew:	-0.117		Prob(JB):	0.626		
Kurtosis:	1.341		Cond. No.	27.0		

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[7]: # Linear model Y ~ X2
model_X2 = ols('Y ~ X2', data=df).fit()
model_X2.summary()
```

/opt/anaconda3/lib/python3.13/site-packages/scipy/stats/_axis_nan_policy.py:430:
UserWarning: `kurtosistest` p-value may be inaccurate with fewer than 20
observations; only n=8 observations were given.

```
[7]:
```

Dep. Variable:	Y	R-squared:	0.408
Model:	OLS	Adj. R-squared:	0.309
Method:	Least Squares	F-statistic:	4.128
Date:	Sun, 21 Sep 2025	Prob (F-statistic):	0.0885
Time:	23:25:58	Log-Likelihood:	-25.100
No. Observations:	8	AIC:	54.20
Df Residuals:	6	BIC:	54.36
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	27.2500	11.608	2.348	0.057	-1.153	55.653
X2	9.2500	4.553	2.032	0.088	-1.891	20.391

Omnibus:	4.296	Durbin-Watson:	0.359
Prob(Omnibus):	0.117	Jarque-Bera (JB):	1.012
Skew:	-0.008	Prob(JB):	0.603
Kurtosis:	1.257	Cond. No.	14.9

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[8]: # ANOVA for both models
from statsmodels.stats.anova import anova_lm

anova_results1 = anova_lm(model1)
anova_results1
```

```
[8]:
```

	df	sum_sq	mean_sq	F	PR(>F)
X1	1.0	231.125	231.125	65.567376	0.000466
X2	1.0	171.125	171.125	48.546099	0.000937
Residual	5.0	17.625	3.525	NaN	NaN

```
[9]: anova_results_X1 = anova_lm(model_X1)
anova_results_X1
```

```
[9]:
```

	df	sum_sq	mean_sq	F	PR(>F)
X1	1.0	231.125	231.125000	7.34702	0.035081
Residual	6.0	188.750	31.458333	NaN	NaN

```
[10]: anova_results_X2 = anova_lm(model_X2)
      anova_results_X2
```

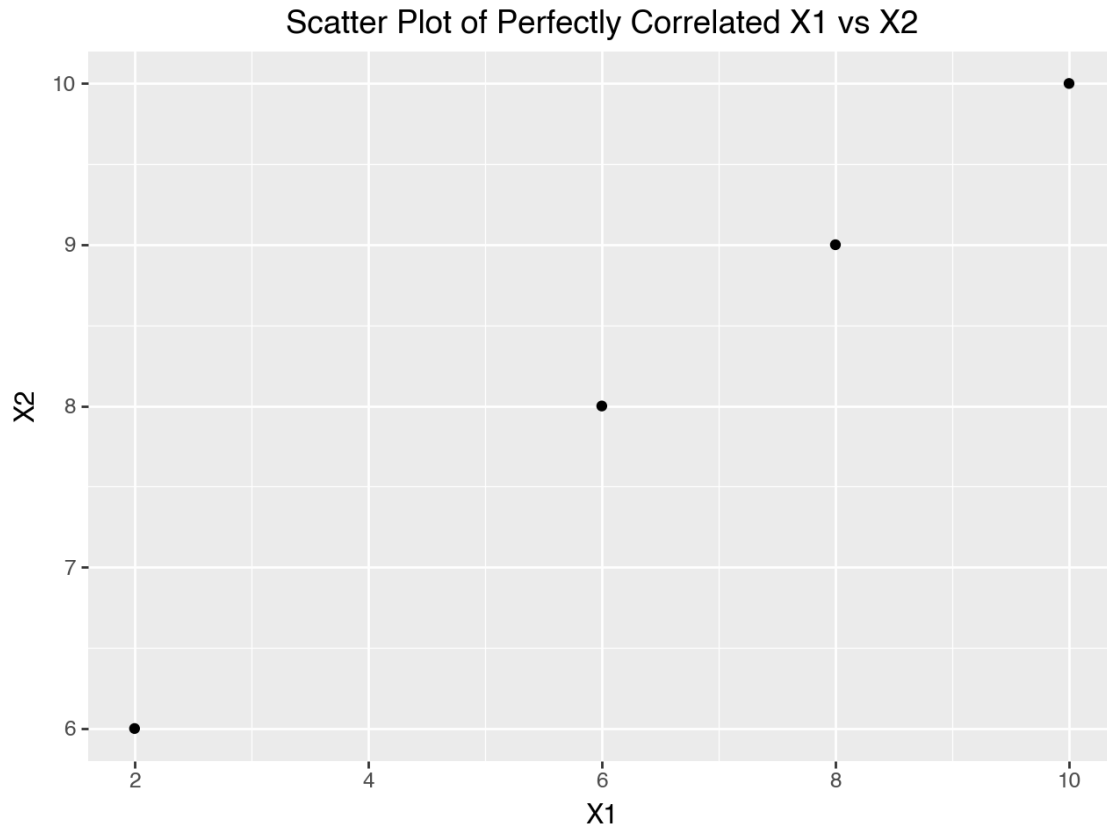
```
[10]:
```

	df	sum_sq	mean_sq	F	PR(>F)
X2	1.0	171.125	171.125000	4.127638	0.08846
Residual	6.0	248.750	41.458333	NaN	NaN

```
[11]: # Perfectly correlated case
      X1 = [2, 8, 6, 10]
      X2 = [6, 9, 8, 10]
      Y = [23, 83, 63, 103]

      df_perfect = pd.DataFrame({
          'X1': X1,
          'X2': X2,
          'Y': Y
      })

      # Plot X1 vs X2
      plot_perfect = (ggplot(df_perfect, aes(x='X1', y='X2')) +
                      geom_point() +
                      labs(x='X1', y='X2', title='Scatter Plot of Perfectly_
↪Correlated X1 vs X2'))
      plot_perfect
```



```
[12]: # Correlation between X1 and X2
correlation_perfect = np.corrcoef(X1, X2)[0, 1]
print(f'Correlation between X1 and X2: {correlation_perfect}')
```

Correlation between X1 and X2: 1.0

```
[13]: X = np.column_stack([np.ones(len(X1)), X1, X2])
X
```

```
[13]: array([[ 1.,  2.,  6.],
           [ 1.,  8.,  9.],
           [ 1.,  6.,  8.],
           [ 1., 10., 10.]])
```

```
[14]: # Matrix operations

XtX = np.dot(X.T, X)
print(f'XtX Matrix:\n{XtX}')
```

XtX Matrix:

```
[[ 4.  26.  33.]
 [ 26. 204. 232.]
```

```
[ 33. 232. 281.]]
```

```
[15]: # Eigenvalues
eigenvalues = np.linalg.eigvals(XtX)
print(f'Eigenvalues:\n{eigenvalues}') #2nd eigen val very close to zero !!!!
#numerically near singular
#this is going to be an issue for us:
```

Eigenvalues:

```
[4.81365468e+02 7.42814262e-17 7.63453185e+00]
```

```
[16]: import scipy.linalg as spla

# Attempt to invert the matrix
XtX_inv = spla.solve(XtX, np.eye(3))
print(f'Inverse of XtX:\n{XtX_inv}')

#it blows up and gives you a warning on the "Condition Number" of the matrix
```

Inverse of XtX:

```
[[ 1.99568144e+16  1.99568144e+15 -3.99136289e+15]
 [ 1.99568144e+15  1.99568144e+14 -3.99136289e+14]
 [-3.99136289e+15 -3.99136289e+14  7.98272577e+14]]
```

```
/var/folders/z7/kfsx1fbn63n3v7l1tt_8lw1gr0000gn/T/ipykernel_32151/3933071815.py:4
: LinAlgWarning: Ill-conditioned matrix (rcond=7.05948e-20): result may not be
accurate.
```

```
[17]: #the warning is basically telling you the result is nonsense. Sanity check!
```

```
[18]: XtX_inv @ XtX # this is not the identity.... cry
```

```
[18]: array([[ 15.    ,  24.    , -9.    ],
 [ 0.125,  13.    ,  1.125],
 [-4.5   , -20.    , -40.5   ]])
```

```
[19]: #If you use np.linalg.inv ... it doesn't do the float calcs exactly and will
      ↪ actually let you invert a singular matrix... watch out!
np.linalg.inv(XtX)

# Interesting discussion of the issue: https://github.com/numpy/numpy/issues/2074
```

```
[19]: array([[ -4.56937087e+15, -4.56937087e+14,  9.13874175e+14],
 [ -4.56937087e+14, -4.56937087e+13,  9.13874175e+13],
 [ 9.13874175e+14,  9.13874175e+13, -1.82774835e+14]])
```

1.1 Variance Inflation Factors - VIFs

```
[20]: # Variance Inflation - generating new data
np.random.seed(0)
X1 = np.random.uniform(25, 50, 100)
X2 = np.random.normal(10, 3, 100)
eps = np.random.normal(0, 4, 100)

df_vif = pd.DataFrame({
    'X1': X1,
    'X2': X2,
    'Y': 7 + 3 * X1 + 5 * X2 + eps
})

# Correlation between X1 and X2
correlation_X1_X2 = np.corrcoef(X1, X2)[0, 1]
print(f'Correlation between X1 and X2: {correlation_X1_X2}')
```

Correlation between X1 and X2: -0.01824782721587361

```
[21]: # Linear model with X1 and X2
mod1 = ols('Y ~ X1 + X2', data=df_vif).fit()
mod1.summary()
```

[21]:

Dep. Variable:	Y	R-squared:	0.978
Model:	OLS	Adj. R-squared:	0.978
Method:	Least Squares	F-statistic:	2193.
Date:	Sun, 21 Sep 2025	Prob (F-statistic):	1.82e-81
Time:	23:26:06	Log-Likelihood:	-277.67
No. Observations:	100	AIC:	561.3
Df Residuals:	97	BIC:	569.2
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	5.9068	2.505	2.358	0.020	0.934	10.880
X1	2.9676	0.055	54.178	0.000	2.859	3.076
X2	5.1591	0.132	39.062	0.000	4.897	5.421

Omnibus:	0.196	Durbin-Watson:	2.109
Prob(Omnibus):	0.907	Jarque-Bera (JB):	0.026
Skew:	0.032	Prob(JB):	0.987
Kurtosis:	3.046	Cond. No.	247.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[22]: vif_model_X1o = ols('X1 ~ X2', data=df_vif).fit()
r2j_X1o = vif_model_X1o.rsquared
vif_j_x1 = 1/(1-r2j_X1o)
```



```
print(f'VIF for X1: {vif_j_x1}')
```

VIF for X1: 1.0003330941128432

1.1.1 VIF Close to 1 indicates not very much multicollinearity between X1 and other predictors.

1.1.2 On the other hand...

```
[23]: # Creating a nearly perfectly collinear variable X3
df_vif['X3'] = 2 * df_vif['X1'] + 3 * df_vif['X2'] + np.random.normal(0, 0.01, 100)

correlation_X1_X3 = np.corrcoef(df_vif['X1'], df_vif['X3'])[0, 1]
correlation_X2_X3 = np.corrcoef(df_vif['X2'], df_vif['X3'])[0, 1]
correlation_comb = np.corrcoef(2 * df_vif['X1'] + 3 * df_vif['X2'], df_vif['X3'])[0, 1]

print(f'Correlation between X1 and X3: {correlation_X1_X3}')
print(f'Correlation between X2 and X3: {correlation_X2_X3}')
print(f'Correlation between (2*X1 + 3*X2) and X3: {correlation_comb}')
```

Correlation between X1 and X3: 0.8464409676819135

Correlation between X2 and X3: 0.5169479332790725

Correlation between (2*X1 + 3*X2) and X3: 0.9999998570072173

```
[24]: # Linear model with X1, X2, and X3
mod2 = ols('Y ~ X1 + X2 + X3', data=df_vif).fit()
mod2.summary()
```

[24]:

Dep. Variable:	Y	R-squared:	0.979
Model:	OLS	Adj. R-squared:	0.978
Method:	Least Squares	F-statistic:	1469.
Date:	Sun, 21 Sep 2025	Prob (F-statistic):	4.67e-80
Time:	23:26:07	Log-Likelihood:	-276.92
No. Observations:	100	AIC:	561.8
Df Residuals:	96	BIC:	572.3
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	5.8079	2.501	2.322	0.022	0.844	10.772
X1	-102.9961	87.588	-1.176	0.243	-276.858	70.866
X2	-153.7766	131.375	-1.171	0.245	-414.553	107.000
X3	52.9822	43.795	1.210	0.229	-33.949	139.914

Omnibus:	0.280	Durbin-Watson:	2.107
Prob(Omnibus):	0.870	Jarque-Bera (JB):	0.044
Skew:	0.020	Prob(JB):	0.978
Kurtosis:	3.094	Cond. No.	4.73e+04

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 4.73e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```
[25]: # VIF calculation for X1
vif_model_X1 = ols('X1 ~ X2 + X3', data=df_vif).fit()
r2j_X1 = vif_model_X1.rsquared
vif_j_X1 = 1 / (1 - r2j_X1)
print(f'VIF for X1: {vif_j_X1}')
```

VIF for X1: 2570128.137094504

```
[26]: # VIF calculation for X2
vif_model_X2 = ols('X2 ~ X1 + X3', data=df_vif).fit()
r2j_X2 = vif_model_X2.rsquared
vif_j_X2 = 1 / (1 - r2j_X2)
print(f'VIF for X2: {vif_j_X2}')
```

VIF for X2: 994491.2150962795

```
[27]: # VIF calculation for X3
vif_model_X3 = ols('X3 ~ X1 + X2', data=df_vif).fit()
r2j_X3 = vif_model_X3.rsquared
vif_j_X3 = 1 / (1 - r2j_X3)
print(f'VIF for X3: {vif_j_X3}')
```

VIF for X3: 3506271.326686703

1.1.3 VIFs > 10 indicate severe multicollinearity among predictors.

```
[ ]:
```

1.2 Exercise

For the dataset “multicollinearity.txt”, perform the following:

1. Check the pairwise correlations of the predictors.
2. Compare the coefficient for each predictor in two cases: a. when regressing Y on that predictor only, and b. when regressing Y on all predictors. Do you see a major change in the coefficients? Their standard errors? What does this tell you about the presence of multicollinearity?
3. Calculate the VIFs for each predictor. Which VIFs are concerning? Comment on what you find.
4. Consider regressing X4 onto the other predictors X1 through X3. Save the residuals from this model and call them `res4`. Then regress Y onto `res4` and look at the coefficient. Compare this coefficient to the coefficients for X4 you found in part 2. What do you notice?
5. Explain geometrically what we are doing in #4. Are we surprised by the results?

```
[1]: import pandas as pd
from statsmodels.formula.api import ols
```

```
from statsmodels.stats.anova import anova_lm
```

```
[2]: multicoldata = pd.read_table('../data/multicollinearity.txt', sep='\s+')
```

```
[3]: multicoldata.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0    y      200 non-null    float64
 1   x1      200 non-null    float64
 2   x2      200 non-null    float64
 3   x3      200 non-null    float64
 4   x4      200 non-null    float64
dtypes: float64(5)
memory usage: 7.9 KB
```

1. Check the pairwise correlations of the predictors.

```
[4]: vars = [ 'x1', 'x2', 'x3', 'x4']

# Calculate the correlation matrix
corr_matrix = multicoldata[vars].corr()
corr_matrix
```

```
[4]:
```

	x1	x2	x3	x4
x1	1.000000	0.999811	0.999495	0.998843
x2	0.999811	1.000000	0.999671	0.999013
x3	0.999495	0.999671	1.000000	0.999306
x4	0.998843	0.999013	0.999306	1.000000

The pairwise correlations of the predictors are all really high. It seems that all 4 predictor variables are all highly correlated.

2. Compare the coefficient for each predictor in two cases: a. when regressing Y on that predictor only, and b. when regressing Y on all predictors. Do you see a major change in the coefficients? Their standard errors? What does this tell you about the presence of multicollinearity?

```
[5]: # Linear model Y ~ X1 + X2 + X3 + X4
model1a = ols('y ~ x1 + x2 + x3 + x4', data=multicoldata).fit()
model1a.summary()
```

```
[5]:
```

Dep. Variable:	y	R-squared:	0.999
Model:	OLS	Adj. R-squared:	0.999
Method:	Least Squares	F-statistic:	7.200e+04
Date:	Wed, 24 Sep 2025	Prob (F-statistic):	8.92e-308
Time:	20:39:59	Log-Likelihood:	-515.82
No. Observations:	200	AIC:	1042.
Df Residuals:	195	BIC:	1058.
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	1.0374	0.462	2.248	0.026	0.127	1.948
x1	-0.1675	0.204	-0.821	0.413	-0.570	0.235
x2	1.1364	0.281	4.050	0.000	0.583	1.690
x3	0.9788	0.256	3.830	0.000	0.475	1.483
x4	1.1175	0.210	5.311	0.000	0.703	1.533

Omnibus:	0.460	Durbin-Watson:	2.103
Prob(Omnibus):	0.795	Jarque-Bera (JB):	0.219
Skew:	-0.046	Prob(JB):	0.896
Kurtosis:	3.133	Cond. No.	377.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[15]: # Linear model Y ~ X1
model_X1a = ols('y ~ x1', data=multicoldata).fit()
model_X1a.summary()
```

[15]:

Dep. Variable:	y	R-squared:	0.999
Model:	OLS	Adj. R-squared:	0.999
Method:	Least Squares	F-statistic:	1.396e+05
Date:	Mon, 22 Sep 2025	Prob (F-statistic):	5.39e-284
Time:	11:45:31	Log-Likelihood:	-589.73
No. Observations:	200	AIC:	1183.
Df Residuals:	198	BIC:	1190.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	1.7257	0.659	2.617	0.010	0.426	3.026
x1	2.1250	0.006	373.575	0.000	2.114	2.136

Omnibus:	0.666	Durbin-Watson:	2.009
Prob(Omnibus):	0.717	Jarque-Bera (JB):	0.451
Skew:	-0.106	Prob(JB):	0.798
Kurtosis:	3.095	Cond. No.	233.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[16]: # Linear model Y ~ X2
model_X2a = ols('y ~ x2', data=multicoldata).fit()
model_X2a.summary()
```

[16]:

Dep. Variable:	y	R-squared:	0.999
Model:	OLS	Adj. R-squared:	0.999
Method:	Least Squares	F-statistic:	1.980e+05
Date:	Mon, 22 Sep 2025	Prob (F-statistic):	5.20e-299
Time:	11:45:54	Log-Likelihood:	-554.80
No. Observations:	200	AIC:	1114.
Df Residuals:	198	BIC:	1120.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	1.3706	0.554	2.473	0.014	0.278	2.464
x2	2.3613	0.005	444.944	0.000	2.351	2.372

Omnibus:	0.045	Durbin-Watson:	1.978
Prob(Omnibus):	0.978	Jarque-Bera (JB):	0.043
Skew:	0.030	Prob(JB):	0.979
Kurtosis:	2.960	Cond. No.	210.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[18]: # Linear model Y ~ X3
model_X3a = ols('y ~ x3', data=multicoldata).fit()
model_X3a.summary()
```

[18]:

Dep. Variable:	y	R-squared:	0.999
Model:	OLS	Adj. R-squared:	0.999
Method:	Least Squares	F-statistic:	2.218e+05
Date:	Mon, 22 Sep 2025	Prob (F-statistic):	6.86e-304
Time:	11:46:28	Log-Likelihood:	-543.45
No. Observations:	200	AIC:	1091.
Df Residuals:	198	BIC:	1098.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	1.0831	0.524	2.066	0.040	0.049	2.117
x3	2.9492	0.006	470.945	0.000	2.937	2.962

Omnibus:	0.792	Durbin-Watson:	2.128
Prob(Omnibus):	0.673	Jarque-Bera (JB):	0.741
Skew:	-0.148	Prob(JB):	0.690
Kurtosis:	2.965	Cond. No.	169.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[19]: # Linear model Y ~ X4
model_X4a = ols('y ~ x4', data=multicoldata).fit()
model_X4a.summary()
```

```
[19]:
```

Dep. Variable:	y	R-squared:	0.998
Model:	OLS	Adj. R-squared:	0.998
Method:	Least Squares	F-statistic:	1.307e+05
Date:	Mon, 22 Sep 2025	Prob (F-statistic):	3.46e-281
Time:	11:46:31	Log-Likelihood:	-596.26
No. Observations:	200	AIC:	1197.
Df Residuals:	198	BIC:	1203.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.8964	0.683	1.312	0.191	-0.451	2.244
x4	4.2002	0.012	361.558	0.000	4.177	4.223

Omnibus:	0.500	Durbin-Watson:	2.234
Prob(Omnibus):	0.779	Jarque-Bera (JB):	0.641
Skew:	0.079	Prob(JB):	0.726
Kurtosis:	2.772	Cond. No.	119.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[22]: anova_results1a = anova_lm(model1a)
anova_results1a
```

```
[22]:
```

	df	sum_sq	mean_sq	F	PR(>F)
x1	1.0	3.004654e+06	3.004654e+06	287798.421928	5.560141e-311
x2	1.0	1.259933e+03	1.259933e+03	120.681635	3.657999e-22
x3	1.0	6.726701e+02	6.726701e+02	64.431163	9.196210e-14
x4	1.0	2.944496e+02	2.944496e+02	28.203621	2.961184e-07
Residual	195.0	2.035826e+03	1.044014e+01	NaN	NaN

```
[23]: anova_results_X1a = anova_lm(model_X1a)
anova_results_X1a
```

```
[23]:
```

	df	sum_sq	mean_sq	F	PR(>F)
x1	1.0	3.004654e+06	3.004654e+06	139558.648159	5.386946e-284
Residual	198.0	4.262879e+03	2.152969e+01	NaN	NaN

```
[24]: anova_results_X2a = anova_lm(model_X2a)
anova_results_X2a
```

```
[24]:
```

	df	sum_sq	mean_sq	F	PR(>F)
x2	1.0	3.005911e+06	3.005911e+06	197975.188289	5.196167e-299
Residual	198.0	3.006288e+03	1.518327e+01	NaN	NaN

```
[25]: anova_results_X3a = anova_lm(model_X3a)
      anova_results_X3a
```

```
[25]:
```

	df	sum_sq	mean_sq	F	PR(>F)
x3	1.0	3.006234e+06	3.006234e+06	221789.259769	6.864799e-304
Residual	198.0	2.683783e+03	1.355446e+01	NaN	NaN

```
[26]: anova_results_X4a = anova_lm(model_X4a)
      anova_results_X4a
```

```
[26]:
```

	df	sum_sq	mean_sq	F	PR(>F)
x4	1.0	3.004367e+06	3.004367e+06	130724.403637	3.458241e-281
Residual	198.0	4.550525e+03	2.298245e+01	NaN	NaN

Summary of coefficients:

regressing Y on all predictors:

x1: B1=-0.1675, SE=0.204, 95% CI range [-0.570, 0.235]

x2: B2=1.1364, SE=0.281, 95% CI range [0.583, 1.690]

x3: B3=0.9788, SE=0.256, 95% CI range [0.475, 1.483]

x4: B4=1.1175, SE=0.210, 95% CI range [0.703, 1.533]

regressing Y on x1 alone: B1=2.1250, SE=0.006, 95% CI range [2.114, 2.136]

regressing Y on x2 alone: B2=2.3613, SE=0.005, 95% CI range [2.351, 2.372]

regressing Y on x3 alone: B3=2.9492, SE=0.006, 95% CI range [2.937, 2.962]

regressing Y on x4 alone: B4=4.2002, SE=0.012, 95% CI range [4.177, 4.223]

There are big changes in the coefficients between the model that includes all predictor variables to the model where it is just one each individual variable. They are not even close and completely differs.

The standard error is also really high for the coefficients in the model that includes all predictors. The standard error in the models where we are regressing Y on only one of the variables drastically reduces the standard error from approx 0.2 to approx 0.006.

3. Calculate the VIFs for each predictor. Which VIFs are concerning? Comment on what you find.

```
[27]: # VIF
      from statsmodels.stats.outliers_influence import variance_inflation_factor
      from patsy import dmatrices #an alternative to basic pandas dfs.

      y, X = dmatrices('y ~ x1 + x2 + x3 + x4',
                       data=multicoldata, return_type='dataframe')

      vif = pd.DataFrame()
      vif["VIF Factor"] = [variance_inflation_factor(X.values, i) for i in range(X.
      ↪shape[1])]
      vif["features"] = X.columns
      print(vif)
```

```
VIF Factor    features
```

```

0      4.080806  Intercept
1  2653.868613      x1
2  4066.198003      x2
3  2162.458352      x3
4   722.288919      x4

```

All of these Variance Inflation Factors (VIFs) are concerning since they are really high (much higher than the typical limit of 10).

4. Consider regressing X_4 onto the other predictors X_1 through X_3 . Save the residuals from this model and call them `res4`. Then regress Y onto `res4` and look at the coefficient. Compare this coefficient to the coefficients for X_4 you found in part 2. What do you notice?

```

[ ]: # Linear model x4 ~ x1 + x2 + x3
model1b = ols('x4 ~ x1 + x2 + x3', data=multicoldata).fit()
model1b.summary()

```

```
[ ]:
```

Dep. Variable:	x4	R-squared:	0.999
Model:	OLS	Adj. R-squared:	0.999
Method:	Least Squares	F-statistic:	4.712e+04
Date:	Wed, 24 Sep 2025	Prob (F-statistic):	7.87e-280
Time:	21:11:04	Log-Likelihood:	-300.24
No. Observations:	200	AIC:	608.5
Df Residuals:	196	BIC:	621.7
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.0994	0.157	0.635	0.526	-0.209	0.408
x1	0.0078	0.069	0.112	0.911	-0.129	0.144
x2	0.0225	0.095	0.236	0.814	-0.165	0.210
x3	0.6626	0.073	9.113	0.000	0.519	0.806

Omnibus:	2.404	Durbin-Watson:	2.129
Prob(Omnibus):	0.301	Jarque-Bera (JB):	1.800
Skew:	-0.004	Prob(JB):	0.407
Kurtosis:	2.535	Cond. No.	358.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

[8]: res4 = model1b.resid
      res4

```

```

[8]: 0    -0.089034
      1     1.792657
      2     0.724258
      3     0.727658
      4     0.033488
      ...

```



```

195    -1.007361
196    -0.034044
197    -1.243194
198    -2.572986
199     1.140979
Length: 200, dtype: float64

```

```
[12]: type(res4)
```

```
[12]: pandas.core.series.Series
```

```
[13]: multicoldata["res4"] = res4
      multicoldata
```

```
[13]:
```

	y	x1	x2	x3	x4	res4
0	1.860	0.374	0.846	1.951	1.325	-0.089034
1	9.747	2.184	3.754	5.099	5.372	1.792657
2	12.707	2.164	3.635	2.505	2.582	0.724258
3	12.822	5.595	4.805	3.653	3.399	0.727658
4	5.160	5.330	2.611	1.873	1.474	0.033488
..	...	...	...	...	...	...
195	413.110	194.952	174.633	140.674	97.743	-1.007361
196	423.084	198.441	180.290	143.616	100.820	-0.034044
197	417.152	196.984	177.431	141.708	98.271	-1.243194
198	418.233	199.412	178.856	144.189	98.636	-2.572986
199	425.688	199.619	180.622	143.935	102.223	1.140979

[200 rows x 6 columns]

```
[ ]: # Linear model Y ~ res4
      model1c = ols('y ~ res4', data=multicoldata).fit()
      model1c.summary()
```

```
[ ]:
```

Dep. Variable:	y	R-squared:	0.000
Model:	OLS	Adj. R-squared:	-0.005
Method:	Least Squares	F-statistic:	0.01938
Date:	Wed, 24 Sep 2025	Prob (F-statistic):	0.889
Time:	21:17:50	Log-Likelihood:	-1245.7
No. Observations:	200	AIC:	2495.
Df Residuals:	198	BIC:	2502.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	215.3599	8.716	24.707	0.000	198.171	232.549
res4	1.1175	8.028	0.139	0.889	-14.714	16.949

Omnibus:	91.438	Durbin-Watson:	0.003
Prob(Omnibus):	0.000	Jarque-Bera (JB):	11.967
Skew:	-0.004	Prob(JB):	0.00252
Kurtosis:	1.802	Cond. No.	1.09

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

coefficients for x4 before:

full model: x4: B4=1.1175, SE=0.210, 95% CI range [0.703, 1.533]

regressing Y on x4 alone: B4=4.2002, SE=0.012, 95% CI range [4.177, 4.223]

coefficient for x4 now:

x4: B4=1.1175, SE=8.028, 95% CI range [-14.714, 16.949]

I see that the coefficient for x4 (β_{x4}) is the same between this $y \sim \text{res4}$ model and the full model we ran before ($y \sim x1 + x2 + x3 + x4$). This is very different value than what we obtained when we looked at regressing y on x4 alone ($y \sim x4$).

5. Explain geometrically what we are doing in #4. Are we surprised by the results?

When we regress x4 onto x1 through x3 ($x4 \sim x1 + x2 + x3$), we are creating a sort of plane that is a linear combination of $x1 + x2 + x3$.

Finding the residuals for this 'res4' compares the actual x4 values with the predicted values and tells us how far off we are with our plane. res4 value are orthogonal to this plane as they are measuring the error between the plane and the actual values. res4 is the part of x4 that is not explained by the combination of $x1 + x2 + x3$.

Then when we regress y onto res4 ($y \sim \text{res4}$), we are doing the exercise of how much does res4 (uniqueness of x4) explain y. This ends up being the same as the x4 part of the full linear regression since x4 was the last variable added to the full linear regression ($y \sim x1 + x2 + x3 + x4$). This is not surprising.